

Data Structures (CIE 1)

– Abstract Data Types, Memory Allocation, Arrays, Linked Lists



1. Module Overview and Exam Focus

- Module 1 covers abstract data types (ADTs), C memory allocation (static, dynamic, stack), and array-based implementations of basic operations (create, insert, traverse, delete, search, update).
- Module 2 introduces linked lists (singly, doubly, circular) and their basic operations with emphasis on pointer manipulation and memory layout.

Exam focus:

- Being able to reason which data structure (array vs linked list variant) is appropriate for a given problem.
- Writing and tracing C code for memory allocation and standard operations on arrays and linked lists.
- Stating and justifying asymptotic time complexity using Big-O notation for these operations.



2. Abstract Data Types (ADTs)

2.1 Formal Notion of an ADT

- An **Abstract Data Type (ADT)** is a mathematical model for a data type that specifies:
 - The *set of values* it can hold.
 - The *set of operations* that can be performed on those values.
 - The *axioms/invariants* that relate operations and values.
- ADTs intentionally hide representation details such as concrete memory layout, pointer structure, or low-level implementation language.

Example (Stack ADT):

- Values: finite sequences of elements of some type.
- Operations: `push(x)`, `pop()`, `top()`, `isEmpty()`, maybe `size()`.
- Laws: after `push(x)`, `top()` returns `x`; after `push(x)` then `pop()`, the stack is restored to its previous state.

2.2 ADT vs Data Structure

- **ADT:** specification level – *what* operations exist and what they mean.
- **Data structure:** implementation level – *how* the ADT is realized in memory using constructs like arrays, pointers, and structs.
- A single ADT can be implemented by multiple different data structures; for example, the Stack ADT can be implemented using either arrays or linked lists.

Example:

- Queue ADT operations: `enqueue`, `dequeue`, `front`, `isEmpty`.
- Implementations: circular array, singly linked list with head/tail pointers.

2.3 Typical ADTs in This Course

- **Sequence-like ADTs:** List, Stack, Queue, Deque.
- **Search ADTs:** Set, Map (Dictionary), Priority Queue.
- For CIE 1, the focus is on basic *sequence-like* ADTs and their implementation using arrays and linked lists.

3. Choosing Appropriate Data Structures

3.1 Key Criteria

When choosing a data structure to implement an ADT, typical criteria include:[4]

- **Access pattern:** random vs sequential access.
- **Update frequency:** frequency of insertions, deletions, and updates.
- **Memory constraints:** fixed vs variable size, contiguous vs scattered allocation.
- **Complexity requirements:** acceptable time complexity for various operations, often expressed using Big-O notation.

3.2 Arrays vs Linked Lists (High-Level)

A high-level comparison as explained for arrays and linked lists:

Aspect	Arrays	Linked Lists
In-language support	Built-in data structure in C	Programmer-defined using struct and pointers
Size	Fixed at compile time for static arrays	Flexible; nodes created and destroyed at runtime
Memory layout	Elements stored contiguously in memory	Nodes can be scattered anywhere in memory; connected by pointers
Random access	Direct access via index in constant time	No direct access; must traverse node by node
Insert/delete in middle	Requires shifting following elements; not constant time	Change a few pointers; constant time once position is known
Per-element overhead	Just data	Data plus at least one pointer field per node

Example:

- An application that frequently inserts in the middle of a long sequence (e.g., a text editor's buffer) is better served by a linked list.
 - An application that frequently accesses elements by index (e.g., random lookups in a table) is better served by an array.
-



4. Memory Allocation Model in C

4.1 Static Memory (Compile-Time Allocation)

- **Static memory** is memory reserved for variables *before* the program runs; this is **compile-time** allocation.
- In C, when an array is declared with a fixed size at file scope or inside a function without dynamic allocation, memory for all its elements is reserved at compile time.
- Capacity cannot grow or shrink at runtime; the size is fixed for the lifetime of the variable.

Example (static array allocation):

```
int students[20];           // space for 20 integers reserved before
                             program runs
printf("%zu\n", sizeof(students)); // typically prints 80 on 64-bit
                             systems
```

4.2 Dynamic Memory (Runtime Allocation)

- **Dynamic memory** is memory allocated *after* the program starts running; this is **runtime** allocation.
- Dynamic memory does not belong directly to a named variable; it is accessed only through pointers.
- In C, dynamic memory is allocated by library functions such as `malloc()` and `calloc()` from `<stdlib.h>`.

Core properties:

- Programmer decides **when** and **how much** memory to allocate.
- Must multiply *number of elements* by `sizeof(type)` when determining allocation size.
- Data initially stored in dynamically allocated memory is **uninitialized** for `malloc()` and **zero-initialized** for `calloc()`.

Example (dynamic allocation for an int array):

```
#include <stdlib.h>

int *students;
int numStudents = 12;

students = calloc(numStudents, sizeof(*students));
printf("%d\n", numStudents * sizeof(*students)); // prints total
bytes, 48 if int is 4 bytes
```

4.3 Stack Memory

- **Stack memory** is a type of dynamic memory reserved for variables declared inside functions.
- When a function is called, stack memory is allocated for its local variables; when the function returns, this memory is automatically freed.
- Excessive recursion or deeply nested calls may exhaust stack memory, causing a **stack overflow**.

Example:

```
void f(void) {
    int a; // array on stack
    // a exists only while f is executing
}
```

4.4 Heap-Allocated Objects and Lifetime

- Memory obtained by `malloc()`/`calloc()` is not automatically freed when a function returns; it remains allocated until explicitly deallocated or program termination.[9]
- Deallocation ("freeing" memory) is performed with `free()` from `<stdlib.h>`. [9]
- Failure to free memory that is no longer needed leads to memory that stays reserved until program end.[9]

Example (allocation and deallocation):

```
#include <stdlib.h>

int main(void) {
    int *arr = malloc(10 * sizeof(*arr));
    if (arr == NULL) {
        // handle error
        return 1;
    }

    // use arr .. arr[9]

    free(arr); // deallocate when no longer needed
    return 0;
}
```

Instance (stack vs heap usage):

- Local temporary buffers that never escape a function (e.g., `int tmp;`) are natural candidates for stack memory.
 - Resizable collections that must survive across function calls are natural candidates for heap allocation via `malloc()`/`calloc()`.
-

5. Arrays in C

5.1 Definition and Declaration

- An **array** in C is a fixed-size collection of elements of the same type stored contiguously in memory.[10]
- General declaration form: `type name[SIZE];` where `SIZE` is a constant expression.

Examples:

```
int myNumbers[] = {25, 50, 75, 100}; // size inferred as 4
int ages[4]; // uninitialized elements
```

- Elements can be initialized in-place with a comma-separated list inside `{}`; all must be of the same type, otherwise truncation or errors occur.

5.2 Array Indexing and Access

- Elements are accessed by zero-based index notation `name[index]`.
- Index 0 denotes the first element, 1 the second, and so on.

Example:

```
int myNumbers[] = {25, 50, 75, 100};
printf("%d\n", myNumbers); // outputs 25

myNumbers = 33; // update element
printf("%d\n", myNumbers); // now outputs 33
```

- Accessing any array element by index is considered a constant-time operation because the element can be looked up directly, regardless of array size.[4]

5.3 Array Size and sizeof Idiom

- The `sizeof` operator gives the total number of bytes occupied by its operand.
- To compute the number of elements in an array: divide total size by size of a single element.

Example:

```
int myNumbers[] = {10, 25, 50, 75, 100};
```

```
int length = sizeof(myNumbers) / sizeof(myNumbers);  
printf("%d\n", length);           // prints 5
```

- This idiom works for arrays of any type, including double and char arrays.

5.4 One-Dimensional vs Multidimensional Arrays

- A **two-dimensional array** can be considered an array of arrays.

Example (2D array):

```
int matrix[3] = { {1, 4, 2}, {3, 6, 8} };  
printf("%d\n", matrix);    // outputs 2  
  
matrix = 9;  
printf("%d\n", matrix);    // now outputs 9
```

- Higher-dimensional arrays (3D and beyond) are declared similarly, e.g. `int arr[4][3];`

5.5 Common Pitfalls

- **Mismatched types in initializer:** using values of different types in the same array leads to truncation or errors.
- **Out-of-bounds access:** indices outside 0 .. n-1 cause undefined behavior; compilers typically do not perform bounds checking for C arrays.

Example of truncation:

```
int myArray[] = {25, 50, 75, 3.15, 5.99}; // 3.15 and 5.99  
truncated to 3 and 5
```

Helpful visual:

- Contiguous layout of array elements in memory: Array as contiguous memory (W3Schools DSA Arrays)
-



6. Array Operations and Algorithms

In this module, arrays are used to implement standard data operations: **create, traverse, search, insert, delete, update**.

6.1 Create

- **Create** an array means allocate storage and optionally initialize elements.
- For static arrays, this is done with a declaration; for dynamic arrays, by allocating memory with `malloc()` or `calloc()`.

Static creation example:

```
#define MAX_SIZE 100
```

```
int arr[MAX_SIZE]; // created at compile time, uninitialized contents
int n = 0;         // current logical size
```

Dynamic creation example:

```
#include <stdlib.h>

int *arr = malloc(MAX_SIZE * sizeof(*arr));
int n = 0;           // track number of used elements
```

6.2 Traverse

- **Traversal:** visiting each element exactly once, typically using a loop.[12]

Example (traversal and printing):

```
void traverse(const int *arr, int n) {
    for (int i = 0; i < n; ++i) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}
```

6.3 Search (Linear Search)

- **Linear search** inspects each element in order and returns the first index where a match is found; if not found, returns -1.

Algorithm (high level):

1. Start from index 0.
2. For each i from 0 to $n-1$, compare $arr[i]$ with the target value.
3. If equal, return i .
4. If loop terminates without a match, return -1.

C implementation:

```
int linear_search(const int *arr, int n, int target) {
    for (int i = 0; i < n; ++i) {
        if (arr[i] == target) {
            return i;    // found
        }
    }
    return -1;          // not found
}
```

Time complexity:

- Best case: target at index 0 $\Rightarrow$ one comparison $\Rightarrow O(1)$.
- Worst case: not found or at last index $\Rightarrow n$ comparisons $\Rightarrow O(n)$.
- Average case (assuming equal likelihood of positions): about $n/2$ comparisons $\Rightarrow O(n)$.

Instance:

- For $arr = [3, 7, 2, 9, 5]$ and target 9, the algorithm checks values at indices 0, 1, 2, 3 and returns 3.

6.4 Insert

- **Insert** into an array at index pos ($0 \leq pos \leq n$) requires:
 - Ensuring capacity ($n < capacity$).
 - Shifting elements from $pos \dots n-1$ one position to the right.
 - Storing the new element at index pos .

C implementation:

```
int insert(int *arr, int *n, int capacity, int pos, int value) {
    if (*n == capacity || pos < 0 || pos > *n) {
        return 0; // failure
    }
    for (int i = *n; i > pos; --i) {
        arr[i] = arr[i - 1]; // shift right
    }
    arr[pos] = value;
    (*n)++;
    return 1; } // success
```

Example:

- Given arr = [10, 20, 30, 40], n = 4, capacity = 5, inserting 25 at pos = 2 results in arr = [10, 20, 25, 30, 40] and n = 5.

6.5 Delete

- Delete** the element at position pos (0-based, $0 \leq \text{pos} < n$) by:
 - Shifting elements from pos+1 .. n-1 one position to the left.
 - Decrementing n.

C implementation:

```
int delete(int *arr, int *n, int pos) {
    if (pos < 0 || pos >= *n) {
        return 0; // failure
    }
    for (int i = pos; i < *n - 1; ++i) {
        arr[i] = arr[i + 1]; // shift left
    }
    (*n)--;
    return 1; } // success
```

Example:

- Deleting element at pos = 1 from [10, 20, 30, 40] gives [10, 30, 40] and n = 3.

6.6 Update

- Update** simply assigns a new value to an existing index, assuming $0 \leq \text{pos} < n$.

C implementation:

```
int update(int *arr, int n, int pos, int newValue) {
    if (pos < 0 || pos >= n) {
        return 0; // failure    }
    arr[pos] = newValue;
    return 1;    // success}
```

Example:

- Updating index 2 in [10, 20, 30, 40] to 35 gives [10, 20, 35, 40].

6.7 Time Complexity of Basic Array Operations

Using Big-O notation as introduced in the time complexity tutorial:

Operation	Description	Time Complexity (worst case)	Justification
Access by index	Read/write arr[i]	$O(1)$	Direct lookup of element at a known index.[4]
Traverse	Visit all elements once	$O(n)$	Loop iterates through all n elements.[11]
Linear search	Find target value by scanning	$O(n)$	May need to inspect each element once.[5]
Insert at arbitrary position	Insert at pos	$O(n)$	Requires shifting up to n-1 elements in worst case.
Delete at arbitrary position	Delete at pos	$O(n)$	Requires shifting up to n-1 elements to fill the gap.
Update	Overwrite arr[i]	$O(1)$	Single direct assignment at known index.